

Experiment #10

Implementation of Inverse Discrete time Fourier Transform

Objective

The objective of this lab is to familiarize student with the discrete time fourier transform and inverse discrete time fourier transform of different signals.

Theory

Discrete Time Fourier Transform:

The discrete-time Fourier transform (DTFT) $X(e^{j\omega})$ of a sequence $x[n]$ is defined

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[k] e^{-j\omega n}$$

In general $X(e^{j\omega})$ is a complex function of the real variable ω and can be written as

$$X(e^{j\omega}) = X_{re}(e^{j\omega}) + jX_{im}(e^{j\omega})$$

where $X_{re}(e^{j\omega})$ and $X_{im}(e^{j\omega})$ are, respectively, the real and imaginary parts of $X(e^{j\omega})$, and are real functions of ω . $X(e^{j\omega})$ can alternately be expressed in the form

$$X(e^{j\omega}) = |X(e^{j\omega})| e^{j\theta(\omega)}$$
$$\theta(\omega) = \arg \{X(e^{j\omega})\}$$

The quantity $|X(e^{j\omega})|$ is called the magnitude function and the quantity $\theta(\omega)$ is called the phase function

In many applications, the Fourier transform is called the Fourier spectrum and, likewise, $|X(e^{j\omega})|$ and $\theta(\omega)$ are referred to as the magnitude spectrum and phase spectrum, respectively.

The DTFT $X(e^{j\omega})$ is a periodic continuous function in ω with a period 2π . The DTFT satisfies a number of useful properties that are often utilized in a number of applications

MATLAB COMMANDS:

For complex Z , the magnitude R and phase angle θ are given by:

$$R = \text{abs}(Z)$$

$$\theta = \text{angle}(Z)$$

$Y = \text{fft}(X)$ returns the discrete Fourier transform of vector X , computed with a fast Fourier transform (FFT) algorithm.

$$Y = \text{np.fft}(X)$$

$Y = \text{fft}(X, n)$ returns the n -point FFT.

$$Y = \text{np.fft}(X, n)$$

$Y = \text{ifft}(X, n)$ returns the n -point IFFT.

$$Y = \text{np.fft.ifft}(X, n)$$

Compute the discrete Fourier transform of the following function analytically and then plot the magnitude and phase:

$$x[n] = 3(0.9)^n u[n]$$

Its DTFT is given as:

$$X(e^{j\omega}) = 3/(1 - 0.9)e^{j\omega}$$

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt

w = np.linspace(0, np.pi, 501) # Generate the frequency vector w

z = np.exp(-1j * w) # Compute z = exp(-j*w)

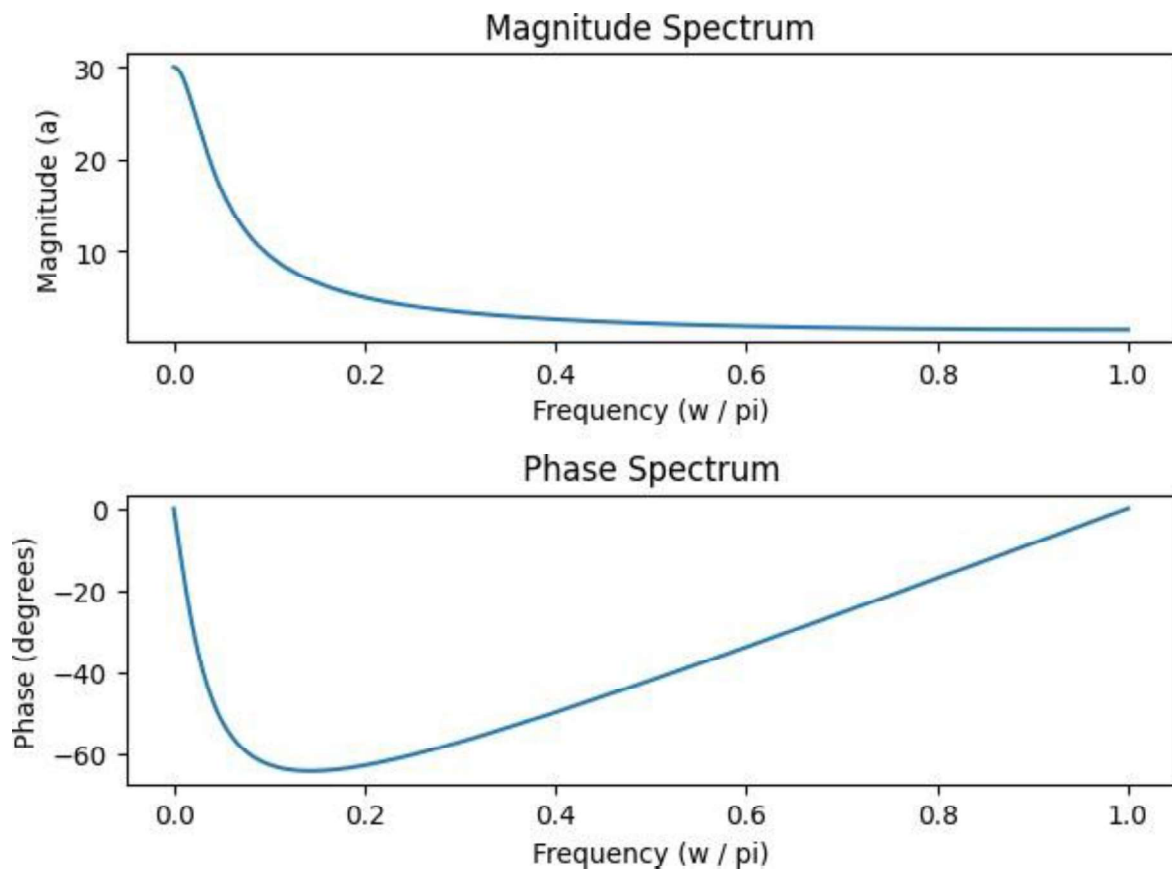
x = 3 * (1 - 0.9 * z)**(-1) # Compute x = 3*(1-0.9*z)^(-1)

a = np.abs(x) # Compute the magnitude of x, a = abs(x)
b = np.angle(x) * 180 / np.pi # Compute the phase angle of x in degrees, b = angle(x)*180/pi

plt.subplot(2, 1, 1)
plt.plot(w / np.pi, a)
plt.xlabel('Frequency (w / pi)')
plt.ylabel('Magnitude (a)')
plt.title('Magnitude Spectrum')

plt.subplot(2, 1, 2)
plt.plot(w / np.pi, b)
plt.xlabel('Frequency (w / pi)')
plt.ylabel('Phase (degrees)')
plt.title('Phase Spectrum')

plt.tight_layout()
plt.show()
```



- Evaluate the DTFT of the given coefficients.

$$\text{num}=[2 \ 1]$$

$$\text{den}=[1 \ -0.6]$$
- Plot real and imaginary parts of Fourier spectrum.
- Also plot the magnitude and phase spectrum.

MATLAB CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import freqz
```

```
# Compute the frequency samples of the DTFT
w = np.arange(-4*np.pi, 4*np.pi + (8*np.pi)/511, 8*np.pi/511)
```

```
num = [2, 1]
den = [1, -0.6]
```

```
h = freqz(num, den, w)
```

```
# Plot the DTFT
plt.subplot(2, 2, 1)
plt.plot(w/np.pi, np.real(h[1]))
plt.grid(True)
plt.title('Real part of  $H(e^{jw})$ ')
```

```

plt.xlabel('w/pi')
plt.ylabel('Amplitude')

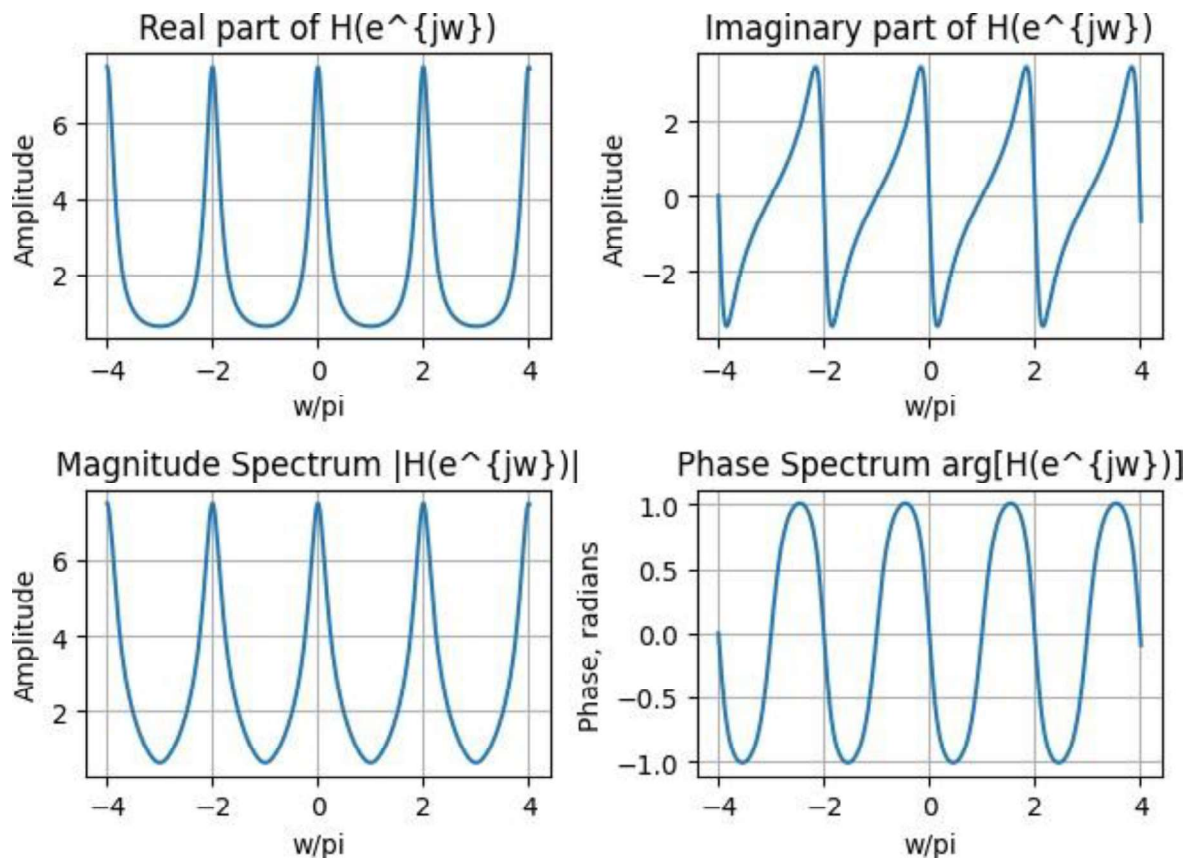
plt.subplot(2, 2, 2)
plt.plot(w/np.pi, np.imag(h[1]))
plt.grid(True)
plt.title('Imaginary part of  $H(e^{jw})$ ')
plt.xlabel('w/pi')
plt.ylabel('Amplitude')

plt.subplot(2, 2, 3)
plt.plot(w/np.pi, np.abs(h[1]))
plt.grid(True)
plt.title('Magnitude Spectrum  $|H(e^{jw})|$ ')
plt.xlabel('w/pi')
plt.ylabel('Amplitude')

plt.subplot(2, 2, 4)
plt.plot(w/np.pi, np.angle(h[1]))
plt.grid(True)
plt.title('Phase Spectrum  $\arg[H(e^{jw})]$ ')
plt.xlabel('w/pi')
plt.ylabel('Phase, radians')

plt.tight_layout()
plt.show()

```



Example

```
import numpy as np
import matplotlib.pyplot as plt
from sympy import symbols, fourier_transform, inverse_fourier_transform

# Define the time vector
t = np.arange(-2*np.pi, 2*np.pi + 0.01, 0.01)

# Define the input signal
M = np.sin(t)

# Plot the input signal
plt.subplot(3, 1, 1)
plt.plot(t, M)
plt.title('Input Signal')
plt.xlabel('t')
plt.ylabel('Amplitude')

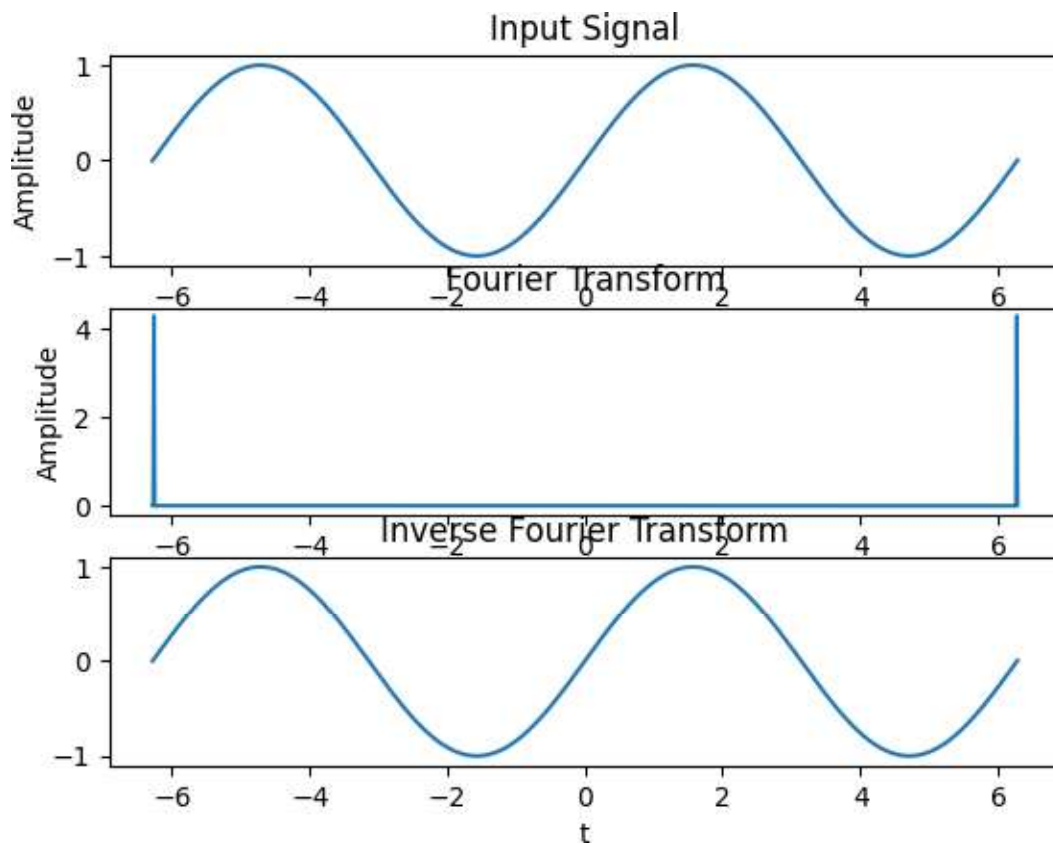
# Compute the Fourier transform
a1 = np.fft.fft(M)

# Plot the Fourier transform
plt.subplot(3, 1, 2)
plt.plot(t, a1)
plt.title('Fourier Transform')
plt.xlabel('t')
```

```
plt.ylabel('Amplitude')

# Compute the inverse Fourier transform
b1 = np.fft.ifft(a1)

# Plot the inverse Fourier transform
plt.subplot(3, 1, 3)
plt.plot(t, b1)
plt.title('Inverse Fourier Transform')
plt.xlabel('t')
```



Lab Tasks

- Suppose that a signal x is given by $x=t*\text{sign}(-3t)$. Write python code to compute the Fourier transform of the signal $X(\Omega)$. Plot the following as well
 - The magnitude of $X(\Omega)$.
 - The angle of $X(\Omega)$.
 - The real part of $X(\Omega)$.
 - The imaginary part of $X(\Omega)$.
- Write python code to compute the Fourier and inverse Fourier transform of signal $x=\cos(t)$.